



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

CFD analysis of a MRL tidal turbine - CFD group project

COMPUTATIONAL FLUID DYNAMICS - ADVANCED
METHODS AND APPLICATIONS

Authors: **Enrico *****, Franco *****, Nicola Filippi**

Contents

Contents	i
1 Introduction	1
2 Numerical model definition	3
2.1 Level of approximation	3
2.2 Mesh generation process	5
2.2.1 Background Mesh	5
2.2.2 SnappyHexMesh	7
2.2.3 Merging meshes	10
2.2.4 Dynamic Mesh definition	11
2.3 Finite Volume discretization of the equations	12
2.3.1 Numerical schemes	12
2.3.2 Numerical solvers	13
2.4 Initial and boundary conditions	14
2.4.1 Initial conditions	14
2.4.2 Boundary conditions	15
2.5 Simulation control	15
3 Results and comments	17
3.1 Reliability assessment of the results	17
3.2 Characteristic curve	19
3.3 Grid independency study	20
3.4 Results comparison	21
3.4.1 Comparison between numerical schemes	21
3.4.2 Comparison between turbulence models	23
3.4.3 Comparison between numerical and experimental results	23
4 Conclusions	27

Bibliography	29
A Appendix A	31

1 | Introduction

The subject of this project is a Momentum Reversal and Lift (MRL) Tidal Turbine (Figure 1.1), which is a cross-flow machine that converts water flow energy into mechanical energy, available in the main shaft of the turbine. As the name of the device suggests, the water flow (renewable) energy is given by tidal currents. Tides are changes in sea level primarily caused by the Moon's shifting gravitational force on Earth's surface as it orbits the planet.

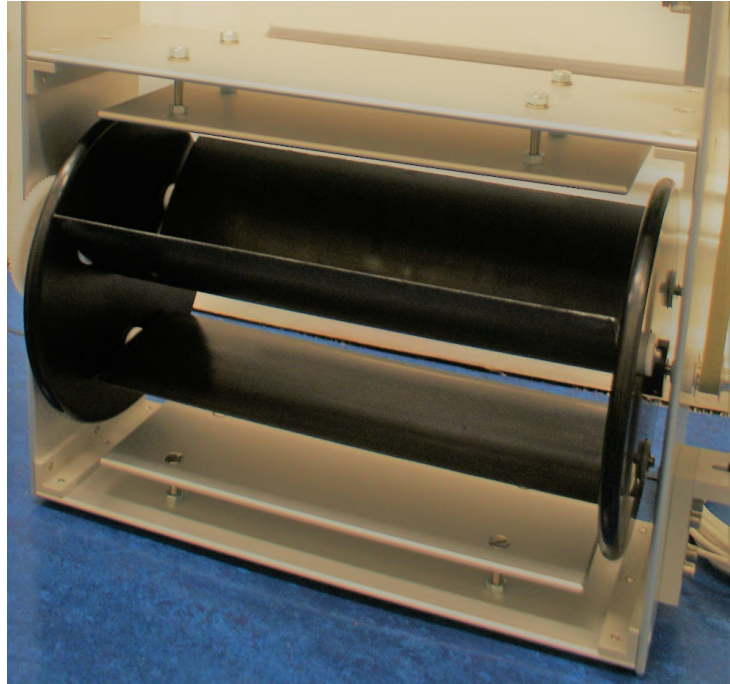


Figure 1.1: MRL tidal turbine [5] similar to the one analyzed during the project.

The 220 mm MRL tidal turbine analyzed in this project has three 70 mm chord-length blades rotating around both their axis (ω_0) and the turbine axis (ω_1). The relation between these two rotation speeds is described by the following equation:

$$\bar{\omega}_1 = -\frac{\bar{\omega}_0}{2} \quad (1.1)$$

As shown in Figure 1.2 the flow-blade interaction can be mainly described by a drag force in the upper part of the turbine and by a lift force when a blade is found in the lower part of the device.

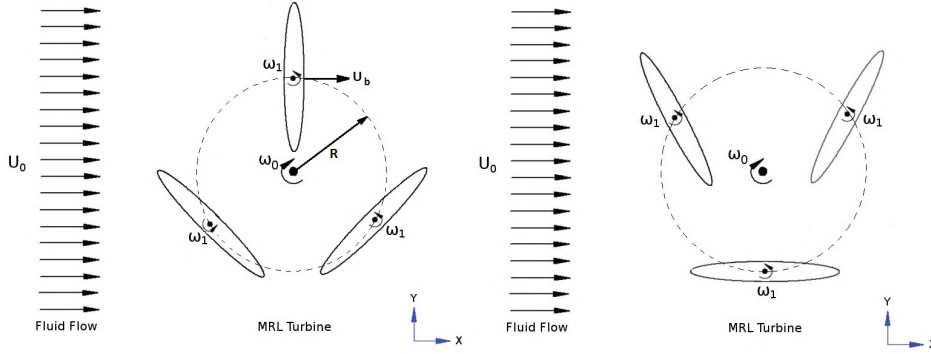


Figure 1.2: (Schematic representation of the turbine motion.)

Considering a free-stream velocity $U_\infty = 1 \text{ m/s}$, the main objectives to achieve in this project were:

- Evaluate the characteristic curve at three different revolutions speeds (20 rpm, 30 rpm and 40 rpm);
- Compare the numerically computed results with experimental data contained in [1];
- Evaluate the behavior of two numerical schemes of different order of accuracy for the divergence term considering a fixed operating point;
- Evaluate the behavior of a second turbulence model for a fixed operating point;

2 | Numerical model definition

2.1. Level of approximation

Water is the Newtonian fluid in which the turbine operates. As the free-stream velocity $U_\infty = 1 \text{ m/s}$ is orders of magnitude smaller than the velocity of sound in water (1480 m/s), corresponding to a Mach number of around $6.8 \cdot 10^{-4} \ll 0.15$, it is reasonable to treat the fluid as incompressible. The constant density of the fluid ρ is 998.7 kg/m^3 while its constant kinematic viscosity ν is $10^{-6} \text{ m}^2/\text{s}$. The water surface and the seabed were not considered in the simulation, hence the turbine was assumed to be in the middle of infinitely deep water.

The characteristic length of the problem was chosen to be the diameter of the turbine (220 mm) fully containing its blades. This means that the Reynolds number for this problem is:

$$Re = \frac{UL_{characteristic}}{\nu} = 2.20 \cdot 10^5 \quad (2.1)$$

The result of (2.1) indicates that the inertial effects are much greater than viscous ones, so the problem is expected to be turbulent.

Turbulence is three-dimensional, but due to the limited computational resources the problem was treated as 2D, neglecting also the border effects of the blades.

Since the problem was modeled using a dynamic mesh, the simulation is unsteady ($\frac{\partial \phi}{\partial t} \neq 0$).

The dynamic description of a viscous fluid is given by the Navier-Stokes equations, which include one mass balance equation, N momentum balance equations (where N corresponds to the dimensions of the space described in the problem) and one energy balance equation. Since the problem is 2D and the fluid is incompressible, the Navier-Stokes equations reduce to (2.2).

$$\begin{cases} \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \bar{U}) = 0 \\ \frac{\partial \rho \bar{U}}{\partial t} + \nabla (\rho \bar{U}) \cdot \bar{U} - \nabla \cdot [\mu(\nabla \bar{U} + \nabla \bar{U}^T)] = \rho \bar{g} - \nabla (P + \frac{2}{3} \mu \nabla \cdot \bar{U}) \end{cases} \quad (2.2)$$

Simplifying (2.2) given that $\rho = c$, $c \in \mathbb{R}^+$, one obtains (2.3).

$$\begin{cases} \nabla \cdot (\bar{U}) = 0 \\ \frac{\partial \bar{U}}{\partial t} + \nabla \bar{U} \cdot \bar{U} - \nabla \cdot [\nu(\nabla \bar{U} + \nabla \bar{U}^T)] = -\nabla p \end{cases} \quad (2.3)$$

As the problem is simplified as a 2D case, which is always a source of error considering the 3D nature of turbulence, and given the limited computational power available, it was decided to use the URANS (Unsteady Reynolds-Averaged Navier-Stokes) equations (2.4), which is the cheapest (computationally) option to describe a turbulent flow.

$$\begin{cases} \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \bar{U}) = 0 \\ \frac{\partial \rho \bar{U}}{\partial t} + \nabla(\rho \bar{U}) \cdot \bar{U} = \rho \bar{g} - \nabla \bar{P} + \mu \nabla^2 \bar{U} + \frac{1}{3} \mu \nabla(\nabla \cdot \bar{U}) - \nabla \cdot (\rho \overline{\bar{U}' \otimes \bar{U}'}) \end{cases} \quad (2.4)$$

The set of equations (2.4) solve (2.2) in a time-averaged manner, so any variable ϕ is rewritten as summation of average $\bar{\phi}$ and fluctuating ϕ' component. This method computes only the largest scale eddies (containing most of the turbulent energy) while modeling the remaining scales, obtaining the highest level of approximation. Moreover, due to the non-linear terms in the momentum balance equations, this method adds 6 extra independent variables, contained in the symmetric Reynolds stress tensor $\bar{\bar{R}} = \rho \overline{\bar{U}' \otimes \bar{U}'}$. To close the problem, first the Boussinesq's hypothesis is implemented to model the deviatoric part of the Reynolds stress tensor:

$$\bar{\bar{R}} = -\rho \frac{2}{3} k \bar{\bar{I}} + 2\mu_T \bar{\bar{D}} \quad (2.5)$$

In this way the isotropic part of $\bar{\bar{R}}$ is only function of the turbulent kinetic energy $k = \frac{1}{2}(\overline{U_x'^2} + \overline{U_y'^2} + \overline{U_z'^2})$, while its deviatoric part is function of the turbulent dynamic viscosity μ_t , reducing the 6 extra-unknowns to only 2. Two ways are typically followed to close the problem:

- Finding a balance equation for one of the two extra variables, while the remaining one is related to it through a calibrated algebraic equation. (One equation models)
- Finding a balance equation for both μ_t and k . (Two equations models)

$k - \omega SST$ (Shear Stress Transport) two-equation turbulence model was used to combine, with a blending factor in the turbulence frequency ω balance equation, the description of the free-shear flow given by $k - \epsilon$ turbulence model and the possibility to solve the near wall region given by $k - \omega$ model. The two-equation $k - \epsilon$ model was discarded due to its

impossibility of resolving the boundary layer (singularity of turbulence dissipation rate ϵ balance equation), essential to well predict flow separation. Spalart-Allmaras one-equation turbulence model was discarded as first choice, even though it can solve the boundary layer, due to lower accuracy in describing flow separation compared to $k - \omega SST$ model [3], this statement will be investigated during the comparison of the results.

The independent variables for the $k - \omega SST$ model are $\bar{U}$ (Velocity), p (pressure), k (turbulent kinetic energy), ω (turbulence frequency) while for the SA model $\bar{U}$, p , $\tilde{\nu}$.

2.2. Mesh generation process

The mesh definition process was divided into four steps:

- background mesh generation;
- castellation, snapping and layer addition for each main region of the fluid domain;
- merging of the meshed regions and 2D mesh conversion;
- creation of the NCCs (Non-Conformal Couplings) and dynamic mesh definition;

2.2.1. Background Mesh

To size the background mesh, it was decided to use multipliers of the characteristic length of the problem, the diameter D of the whole turbine (Figure 2.1).

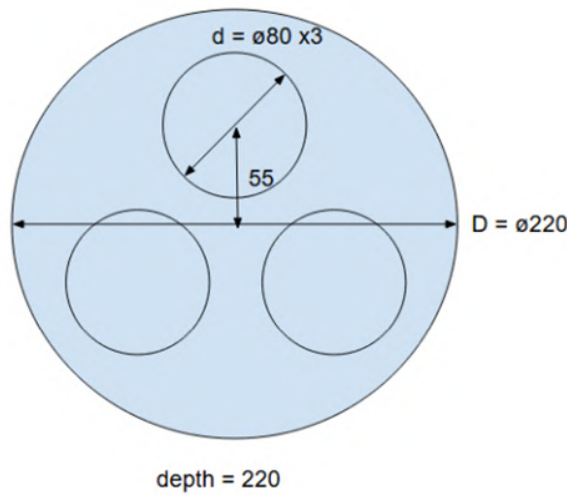


Figure 2.1: Dimensions of the MRL tidal turbine (all values in mm)

The computational domain was modeled as a rectangular cuboid with horizontal edges

aligned with the inflow. As can be seen in Figure 2.2, the inlet, top wall and bottom wall of the domain were set at 5 diameters D from the turbine, assuming to be sufficient to minimize the influence of artificial boundary conditions on the simulation results. For the downstream length, it was decided to use 10 diameters, assuming this is enough to not influence the results. This was checked by comparing them to the ones obtained from a fluid domain of double the size. The patches of the mesh are defined as in Figure 2.2: the back and front walls were considered empty (2D case), the top and bottom immaterial boundaries were set to be symmetry patches, while the left and right vertical walls were the inlet and outlet of the simulation respectively.

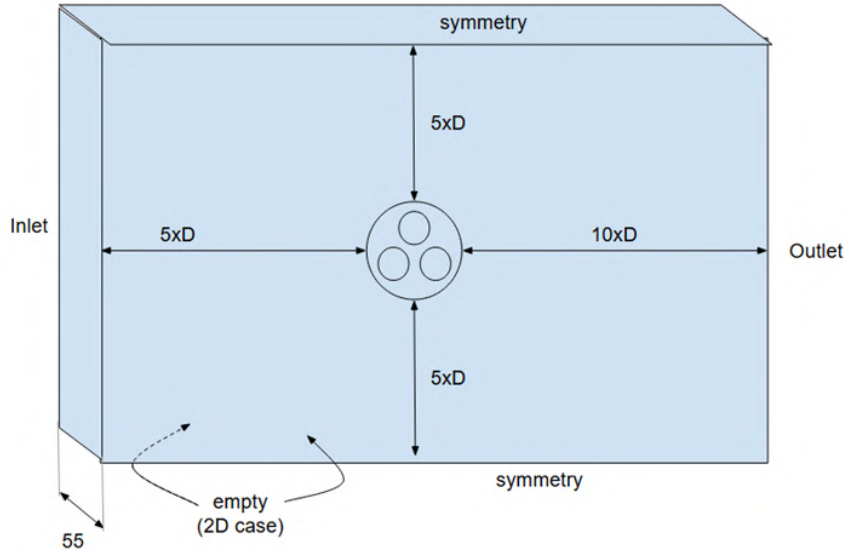


Figure 2.2: Schematic representation of the fluid domain, its dimensions and the type of boundaries applied.

The blockMesh utility was used to create the structured background mesh, the dimensions of the domain (already mentioned), together with the mesh size are shown in Figure 2.3.

As described in Figure 2.3 the depth of the domain is equal to the characteristic cell size; this was done to both reduce the number of cells generated in the unresolved dimension by the snappyHexMesh utility, and to get a cell aspect ratio equal to one, improving convergence of the snapping procedure (as stated in the snappyHexMesh utility documentation).

The calculation grid generated by the blockMesh utility, following the settings shown in Figure 2.3 is displayed in the following Figure 2.4.

```

convertToMeters 1;

dim // a subdictionary containing all the variables needed for the creation of the mesh
{
    minX -1.21; // 5 diameters of the whole turbine upstream
    maxX 2.31; // 10 diameters of the whole turbine downstream
    minY -1.21; // 5 diameters of the whole turbine on bottom
    maxY 1.21; // 5 diameters of the whole turbine on top
    minZ 0.0;
    maxZ 0.055; // to obtain one-cell-thick domain

    xElemDim 0.055; // element size along x-direction
    yElemDim 0.055; // element size along y-direction
    zElemDim 0.055; // element size along z-direction

    nELX #calc "std::ceil(($maxX - $minX) / $xElemDim)"; // using the standard library to round up the cell number
    nELY #calc "std::ceil(($maxY - $minY) / $yElemDim)";
    nELZ #calc "std::ceil(($maxZ - $minZ) / $zElemDim)";
}

vertices
(
    ($dim/minX $dim/minY $dim/minZ)
    ($dim/maxX $dim/minY $dim/minZ)
    ($dim/maxX $dim/maxY $dim/minZ)
    ($dim/minX $dim/maxY $dim/minZ)
    ($dim/minX $dim/minY $dim/maxZ)
    ($dim/maxX $dim/minY $dim/maxZ)
    ($dim/maxX $dim/maxY $dim/maxZ)
    ($dim/minX $dim/maxY $dim/maxZ)
);

blocks
(
    hex (0 1 2 3 4 5 6 7) ($dim/nELX $dim/nELY $dim/nELZ) simpleGrading (1 1 1)
);

```

Figure 2.3: piece of blockMeshDict used to create the background mesh.

2.2.2. SnappyHexMesh

Since the primary focus of the analysis is the interaction between the MRL tidal turbine and the water flow, mesh refinement around the turbine was necessary to accurately capture the conversion of flow energy into mechanical power.

The snappyHexMesh utility was used to manually refine and add layers separately in each main region (Figure 2.5) of the fluid domain. The refinement wasn't applied on the complete domain, as in that way, it was not possible to create layers on the internal patches representing the NCC interface. A structured layered mesh was preferred on those faces to decrease as much as possible the interpolation error of the NCC.

In Figure 2.6 are represented the refinement regions added in the snappyHexMesh utility. Refinement region 1 is the background mesh (coarsest mesh) because the flow is expected to be almost undisturbed by the turbine. To achieve a proper refinement around the turbine while avoiding a sudden change in mesh size, so limiting the skewness and non-orthogonality close to the turbine, three box refinements were implemented (region 2 to 4) with a refinement level which went from 1 to 3 towards the turbine. Region 5 (AMI-ext) was filled with cells following a level 4 cell splitting as well as surface refined with the same level to achieve a better compatibility between moving parts of the mesh (limiting the interpolation error). Region 6 (AMI-int*) of the mesh was both surface and region refined following a level 5 cell splitting, for the same reasons given for region 5. Finally, the three blades (blade*) contained in region 6 were surface refined following a level 7

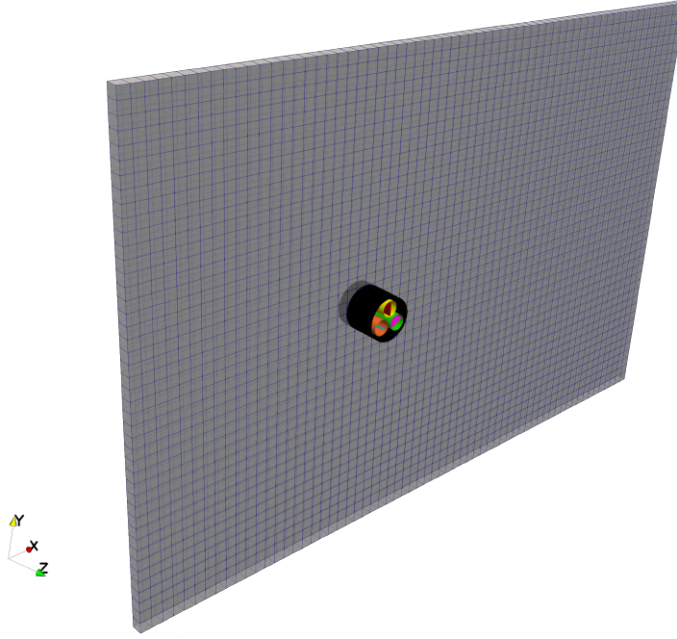


Figure 2.4: Background mesh and the geometries used to describe the MRL tidal turbine.

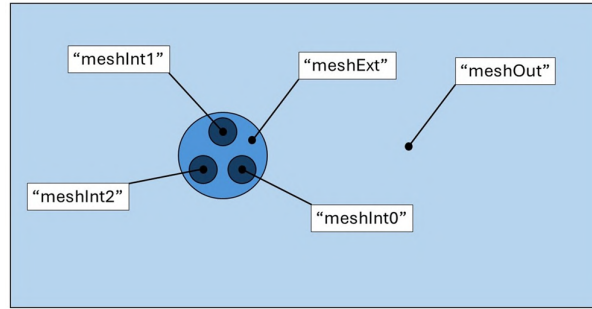


Figure 2.5: nomenclature of the five regions of the fluid domain that were meshed separately.

cell splitting. In the **first mesh**, to achieve $y^+ \approx 1$ at the blade wall, it was used the following layer mesh configuration:

- expansion ratio of 1.667 for the first extruded layer.
- expansion ratio of 1.65 and final layer thickness of $0.6 \cdot 0.61 = 0.364$ for the next 7 extruded layers.

This sequence of layer extrusion seems unnecessary, but it was essential to make independent the number of layers on the NCC interface from the one on the blades, and the in the case of the second (refined) mesh was used to achieve both a $y^+ \approx 1$ (which corresponded to a first layer height of around $7.75 \mu m$) and a layered mesh height sufficient to capture the viscous and buffer layer, which was found to be approximately $6.4 \cdot 10^{-4} m$.

The decision to solve the boundary layer is justified by the fact that a flow separation is expected on the blades due to the eventual extremely large angle of attack and due to the need of solving the boundary layer to compute the wall-shear stress and drag force with good precision, as it is one of the two loads that will determine the torque applied on the turbine, so it's performance.

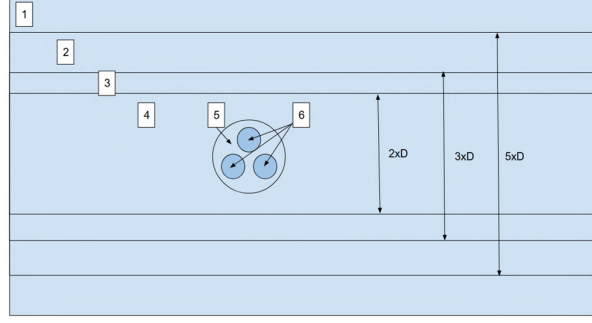


Figure 2.6: Refinement regions defined in the snappyHexMesh dictionary.

The **second mesh** was a refined version of the first one. The refinement was achieved by simply changing the background mesh cell size from 0.055 m to 0.0275 m. Moreover, the parameters controlling the layers at the blades wall were changed to (for the already mentioned reasons):

- expansion ratio of 1.25 for the first extruded layer.
- expansion ratio of 1.37 and final layer thickness of $0.8 \cdot 0.73 = 0.584$ for the next 10 extruded layers.

The **third mesh** was one level coarser than the first one, as the number of cells in the second mesh (before merging) was already high ($13.8 \cdot 10^6$ cells), and expecting a cubic growth in cell count, 64 GB of Random Access Memory wasn't sufficient for another level of refinement, assuming that 2 GB of it are needed to temporarily store 10^6 cells. Even if the extrudeMesh utility could have been used before the merging procedure to reduce the maximum cell count, the amount of hours ($\approx 25 h_{clock}$ or $\approx 200 h_{CPU}$) that it took to complete 1.5 s of simulation with the second mesh was already large for the limited time available. Therefore, the background mesh cells size was increased from 0.055 m (first mesh) to 0.11 m, while the parameters controlling the layers at the blades wall were changed to:

- expansion ratio of 1.667 for the first extruded layer.
- expansion ratio of 1.6 and final layer thickness of $0.6 \cdot 0.625 = 0.375$ for the next 9 extruded layers.

These parameters were different for blade 1 of the 3rd mesh, as it wasn't possible to extrude the layers at its tips, so a level 9 (instead of 7) surface refinement was used to solve the issue.

2.2.3. Merging meshes

After meshing each main region of the fluid domain (Figure 2.5), the complete calculation grid was created by merging the regions together using "mergeMeshes" utility. It is necessary to mention that after the meshing of the regions "meshExt" and "meshInt*", the topoSet utility was used to create, first the cell-set (using a "cylinderToCell" source which contained the entire region), and by conversion, the cell-zone used to control the mesh relative motions during simulation.

After obtaining the entire meshed fluid domain, to convert the complete 3D mesh into a 2D resolving mesh and to set up the NCC between regions interested by relative motion these steps were followed:

- with the changeDictionary utility the boundary types were changed from patches to the one also described in Figure 2.2;
- with extrudeMesh utility the 2D mesh on the front face of the domain was extruded linearly by one cell so to comply with the 2D domain definition in OpenFOAM;
- with createNonConformalCouple utility the NCCs were created between "meshOut" and "meshExt", as also between all three "meshExt" - "meshInt*" interfaces. The creation of face zones wasn't necessary as the boundaries of the single regions were sufficient (the boundaries sharing the same patch were called differently to not let mergeMeshes utility join them in a single boundary);
- renumberMesh utility was implemented to make the computations more efficient by renumbering the cells following Cuthill-McKee algorithm, decreasing the bandwidth and profile of the matrix of coefficient, and improving the linear solver convergence speed.

After this step the quality of the three meshes is analyzed using the built-in checkMesh utility of OpenFOAM, the output of which is summarized by the following Table 2.1.

In Figure 2.7 is shown the complete mesh number 1.

	Mesh 1	Mesh 2 (refined)	Mesh 3 (coarsest)
cell count	71340	240311	33701
Max. aspect ratio	52.0	32.2	41.2
Non-orthogonality	37.9°(max), 5.2°(avg.)	42.4°(max), 4.0°(avg.)	49.2°(max), 6.3°(avg.)
Max. skewness	1.28	0.95	1.3
checkMesh errors	0	0	0

Table 2.1: mesh quality parameters obtained for the three main meshes.

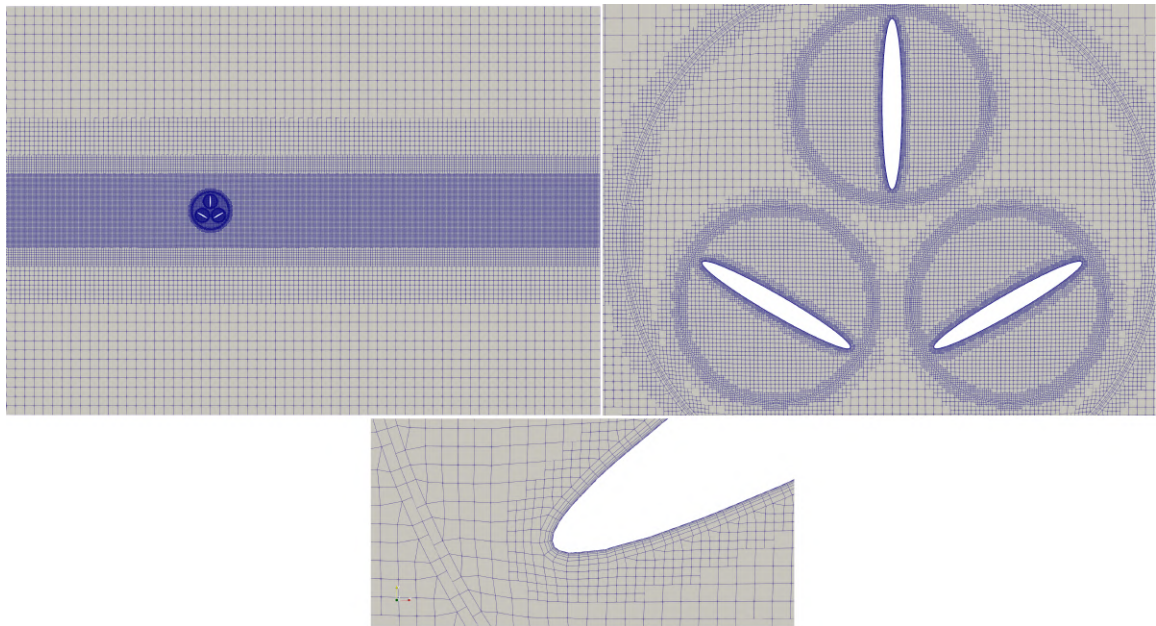


Figure 2.7: the complete 2D mesh1 (top-left), a detailed view of the turbine mesh refinements (top-right) and of the blade (bottom-center)

2.2.4. Dynamic Mesh definition

The previously defined cell zones were essential for specifying the mesh regions subjected to relative rotational motion. These motions were defined in the `dynamicMeshDict`, part of which are displayed in the Figure 2.8.

```

rpm_T -40;
rpm_b 20;

mover
{
    type            motionSolver;
    libs            ("libfvMeshMovers.so" "libfvMotionSolvers.so");
    motionSolver    motionSolverList;

    solvers
    {
        rot-ext
        {
            motionSolver    solidBody;
            cellZone        rot-ext;
            solidBodyMotionFunction    rotatingMotion;

            origin    (0 0 0);
            axis      (0 0 1);
            rpm        $rpm_T;
        }
    }
}

rot-int0
{
    motionSolver    solidBody;
    cellZone        rot-int0;
    solidBodyMotionFunction    multiMotion;

    rotating0 // rotating with the turbine
    {
        solidBodyMotionFunction    rotatingMotion;
        rotatingMotionCoeffs
        {
            origin    (0 0 0);
            axis      (0 0 1);
            rpm        $rpm_T;
        }
    }
    rotating1 // rotating around its axis
    {
        solidBodyMotionFunction    rotatingMotion;
        rotatingMotionCoeffs
        {
            origin    (0.0476313972 -0.0275 0);
            axis      (0 0 1);
            rpm        $rpm_b;
        }
    }
}

```

Figure 2.8: definition of the "meshExt" region rotation (left) and of the "meshInt0" region rotations (right), from the dynamicMeshDict.

2.3. Finite Volume discretization of the equations

2.3.1. Numerical schemes

The numerical discretization methods used for the terms of the balance equations are summarized in the Table 2.2.

Term/operator to discretize	Chosen numerical scheme
Time derivative	Euler (1^{st} order accu. in time)
Gradient	Gauss Linear and cellLimited Gauss Linear 1 (U)
Divergence(ϕ, U)	bounded Gauss linearUpwind grad(U)
Divergence(ϕ, k)	Gauss limitedLinear 1
Divergence(ϕ, ω)	Gauss limitedLinear 1
Divergence of viscous stress tensor (deviatoric)	Gauss linear
Laplacian	Gauss linear limited corrected 0.33
Interpolation	Linear
Surface normal Gradient Schemes	limited corrected 0.33
nearest wall distance	meshWave method

Table 2.2: numerical schemes used to discretize the terms of the balance equations.

The 2^{nd} order numerical scheme "bounded Gauss linearUpwind" used to discretize the divergence of the velocity field, was used instead of "bounded Gauss limitedLinear 1" (potentially 2^{nd} order) as the latter required really small relaxation factors of p and U (from 0.6 down to 0.005 for larger Δt) or time-step-size to achieve the same specified residual of

the pressure equation at each time-step, increasing the number of outer iterations, so the convergence speed. As one can imagine, "bounded Linear" numerical scheme was even more unstable than "limitedLinear".

For the Laplacian discretization the Gauss linear limited corrected with coefficient 0.33 was chosen, as the maximum orthogonality is around 40° , a non-negligible amount, but not even severe, so it is corrected in a less intense way and it's bounded to increase stability.

2.3.2. Numerical solvers

As the simulated time was at least 1.5 s and a fine time-resolution wasn't necessary to measure the performance of the fluid machine, it was decided to use the PIMPLE algorithm to solve the RANS equations and the two extra equations (k and ω), which is a mix between the SIMPLE and PISO algorithm, doing X outer iterations, inside of which 1 or more inner (PISO) iterations are also computed. The matrix of the coefficient of the semi-discretized momentum equation is updated only at the beginning of a new outer iteration. The settings used for the PIMPLE algorithm are described in Figure 2.9.

```
PIMPLE
{
    correctMeshPhi          yes;

    nNonOrthogonalCorrectors 0;
    nOuterCorrectors         25; // outer iterations
    nCorrectors              1; // inner (PISO) iterations

    outerCorrectorResidualControl
    {
        p
        {
            tolerance 1e-5;
            relTol    0;
        }
    }
}

relaxationFactors
{
    fields
    {
        p          1;
        pFinal     1;
    }

    equations
    {
        U          1;
        k           1;
        omega       1;

        "(U|k|omega)Final" 1;
    }
}
```

Figure 2.9: parameters used for the PIMPLE algorithm.

The linear solvers used are summarized in Table 2.3.

Syst. of equations	Solver	Abs. Tol.	Rel. Tol.
p	GAMG	10^{-8}	10^{-2}
pFinal	GAMG	10^{-8}	0
pcorr.	GAMG	10^{-4}	0
MeshPhi	smoothSolver (symG-S)	10^{-2}	0
U k	smoothSolver (symG-S)	10^{-9}	10^{-3}
(U k)Final	smoothSolver (symG-S)	10^{-9}	0
ω	smoothSolver (symG-S)	10^{-10}	10^{-3}
ω_{Final}	smoothSolver (symG-S)	10^{-10}	0

Table 2.3: linear solvers used for each equation of the problem.

2.4. Initial and boundary conditions

2.4.1. Initial conditions

It was decided to use as initial conditions the resulting fields of the steady-state simulation (SIMPLE algorithm) of the fixed tidal turbine, by implementing the mapFields utility. In this way it was saved the computational time that would be necessary to reach a steady periodic trend of the flow quantities (Figure 2.10) and also improving the initial convergence of the algorithm.

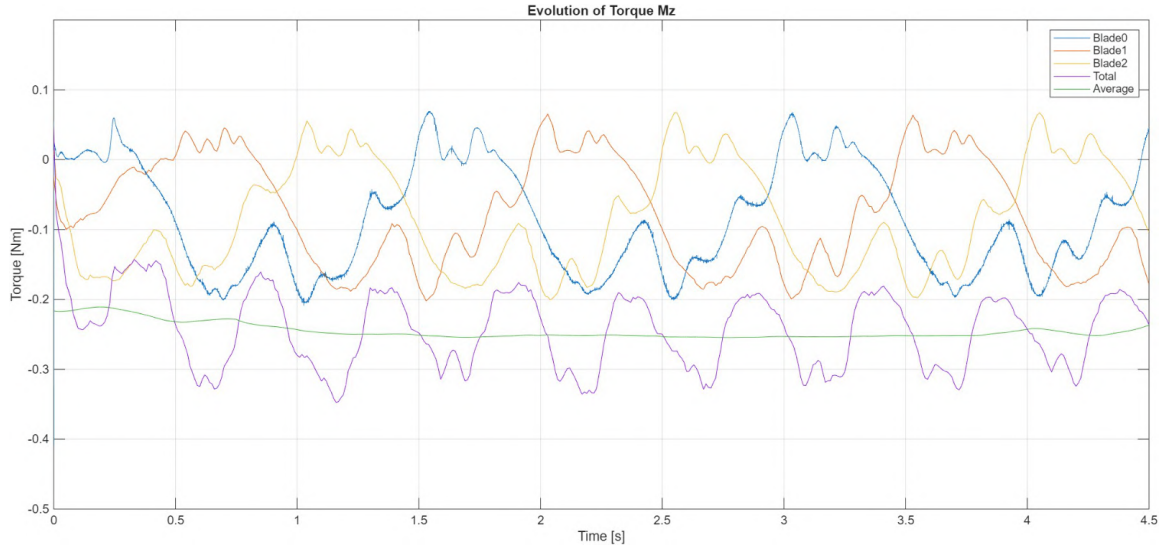


Figure 2.10: trend of torque for the first three full cycles (1.5 s per cycle) of the turbine, starting from a steady state solution.

2.4.2. Boundary conditions

The inlet velocity is $U = 1 \text{ m/s}$ and the inlet turbulence has a 3% intensity, as it was measured experimentally [1]. The turbulence length scale was set to be a fraction of the characteristic length D (220 mm), namely $1/10 \text{ } D$. From this data, it was possible to compute an estimated value of the turbulent variables at the inlet of the domain for the use of $k - \omega SST$ two-equation model:

$$k = \frac{3}{2} U^2 I^2 = 0.00135 \text{ m}^2/\text{s}^2 \quad (2.6)$$

$$\omega = 0.09^{-0.25} \frac{\sqrt{k}}{l_t} = 3.049 \text{ s}^{-1} \quad (2.7)$$

In the following Table 2.4, the boundary condition type for each variable is presented. The front and back patches were set to empty, while the top and bottom ones to symmetry. The normalized pressure p at the outlet was set to 0.

	Inlet	Outlet	Blades	AMI
U	fixedValue	inletOutlet	movingWallVelocity	movingWallSlipVel.
p	zeroGradient	fixedValue	zeroGradient	zeroGradient
ν_t	calculated	zeroGradient	nutUWallFunction	zeroGradient
k	fixedValue	inletOutlet	kqRWallFunction	zeroGradient
ω	fixedValue	inletOutlet	omegaWallFunction	zeroGradient

Table 2.4: types of boundary condition set for all the variables.

For the use of Spalart-Allmaras one-equation turbulence model, the boundary conditions are equivalent for U , p , while ν_t and the new variable $\tilde{\nu}$ had a different boundary condition type at the inlet, fixedValue (also at the blades, only for $\tilde{\nu}$). These inlet values were computed according to [2] which included as reference also [4]:

- 4 times the molecular viscosity for $\tilde{\nu}$;
- 0.75 times the molecular viscosity for ν_t ;

2.5. Simulation control

The foamRun application with the incompressibleFluid solver was used to solve the problem. For the unsteady simulation, the initial time-step-size was set to $5 \cdot 10^{-6} \text{ s}$, with the possibility of dynamically adjusting it based on the set Courant number Co . This parameter was not the same for all simulations, as the optimal value, in terms of simulation

solving speed, changed based on the angular velocity of the turbine. For example the optimal value for the 40 rpm case was $Co=0.5$ (the maximum Courant number used in all simulations), as shown in Figure 2.11.

The function objects used in the simulation were:

- "yPlus", to get the maximum value of y^+ on each blade, adjusting the layered mesh accordingly (not dynamically);
- "forcesIncompressible", three of them were used to compute the torque generated at the turbine centre by each blade interacting with the flow;
- "wallShearStress", to visualize flow separation and compute the viscous forces on each blade.

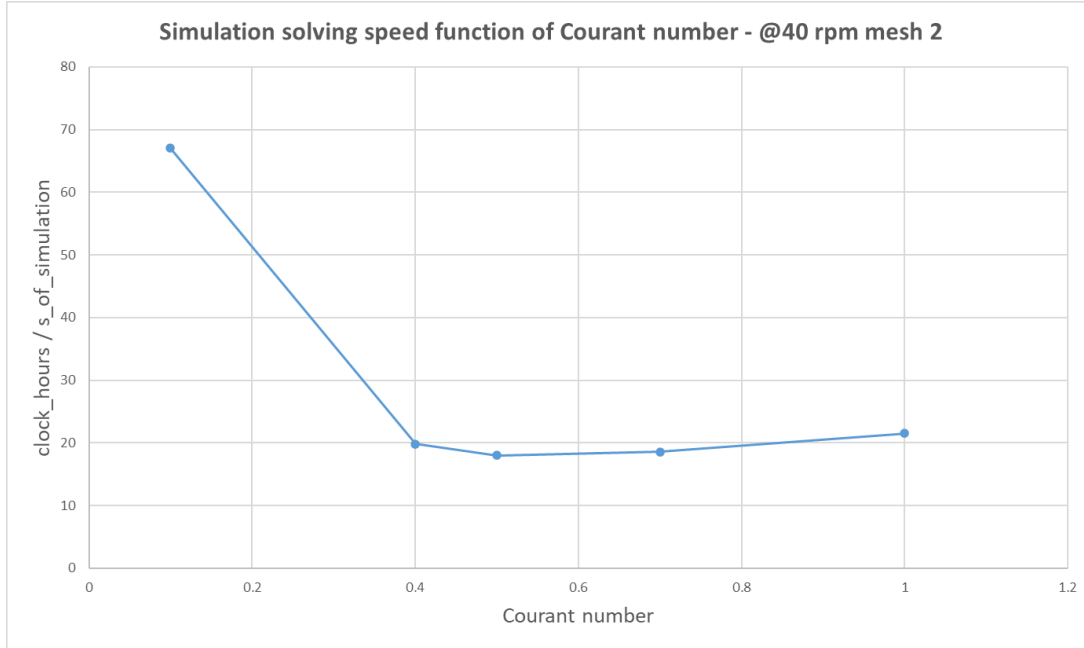


Figure 2.11: simulation solving speed (h/s) function of the Courant number for the 40 rpm case, starting from the steady-state solution.

3 | Results and comments

3.1. Reliability assessment of the results

Before analyzing the results, the y^+ and the initial residuals were analyzed to check if these values were within the set limits: close to 1 for the y^+ and lower than 10^{-5} for the residuals. The trend of the maximum y^+ obtained on the blades for the first mesh is reported in Figure 3.1, for the two remaining meshes the maximum value were compatible: 1.08 (2^{nd} mesh) and 1.09 (3^{rd} mesh).

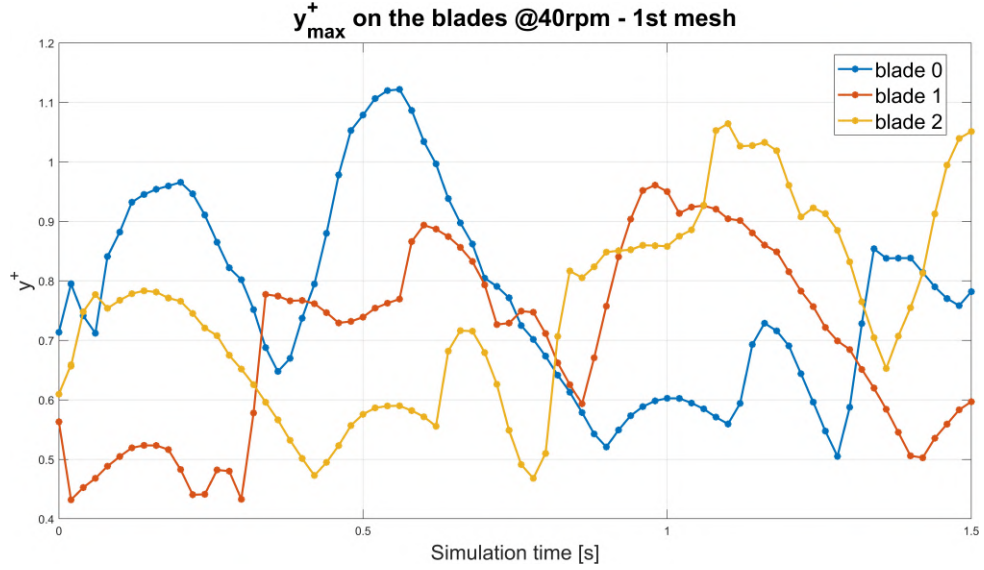


Figure 3.1: maximum value of y^+ on all three blades during the full cycle of the turbine.

In the first trials of simulation it was found the right value for the first layer height ($y^+ \approx 1$) and the layered mesh height (containing viscous and buffer layers), already reported in subsection 2.2.2. These values were qualitatively checked by plotting the trend of the tangent velocity along the layered mesh height in the most critical cell and time-step (Figure 3.2). This was done in the open source software ParaView following the steps:

- applying the "PointDatatoCellData", to avoid the linear interpolation between cell

centers;

- using the ruler, it was found the inclination angle of the critical (y_{max}^+ of all three blades) cell;
- using the calculator it was created a unit vector in the x-y plane with the direction given by the inclination found at the previous step (locally tangent to the blade);
- again, using the calculator it was found the scalar product between U and the tangent vector, giving the tangent velocity.

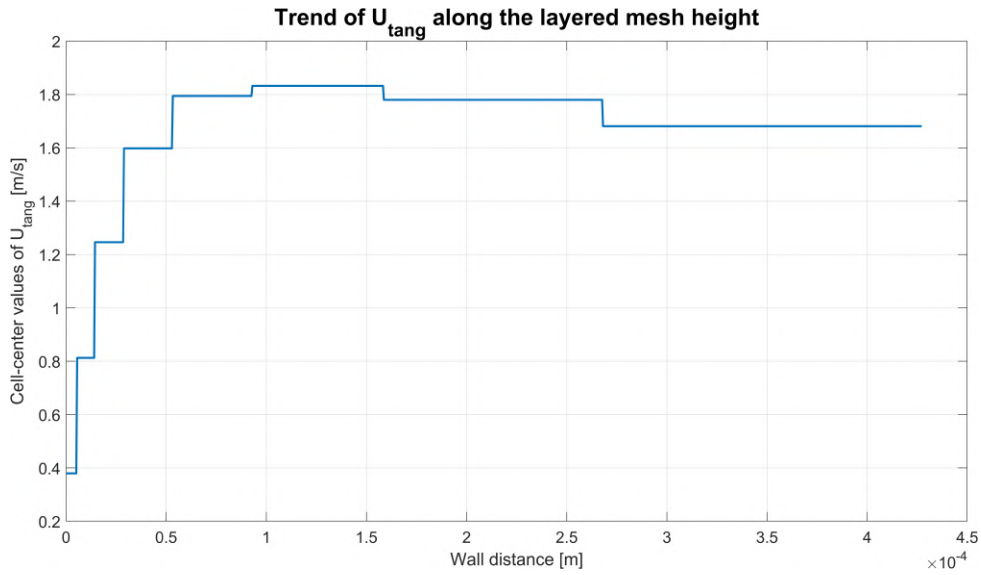


Figure 3.2: trend of the cell-center tangent velocity along the layered mesh height, in the most critical cell (y_{max}^+) and time-step.

In Figure 3.2 can be distinguished the viscous (linear), buffer and logarithmic layers, qualitatively validating the mesh parameters used.

The initial residual of the pressure equation was the slowest and most challenging to converge, so it was taken as a reference to check convergence of each time-step. As an example, in Figure 3.3 are shown the pressure initial residuals for both the chosen "linearUpwind" and "limitedLinear 1" numerical schemes discretizing the divergence of velocity, highlighting the difference in stability and the reason why the latter was discarded (explained in subsection 2.3.1).

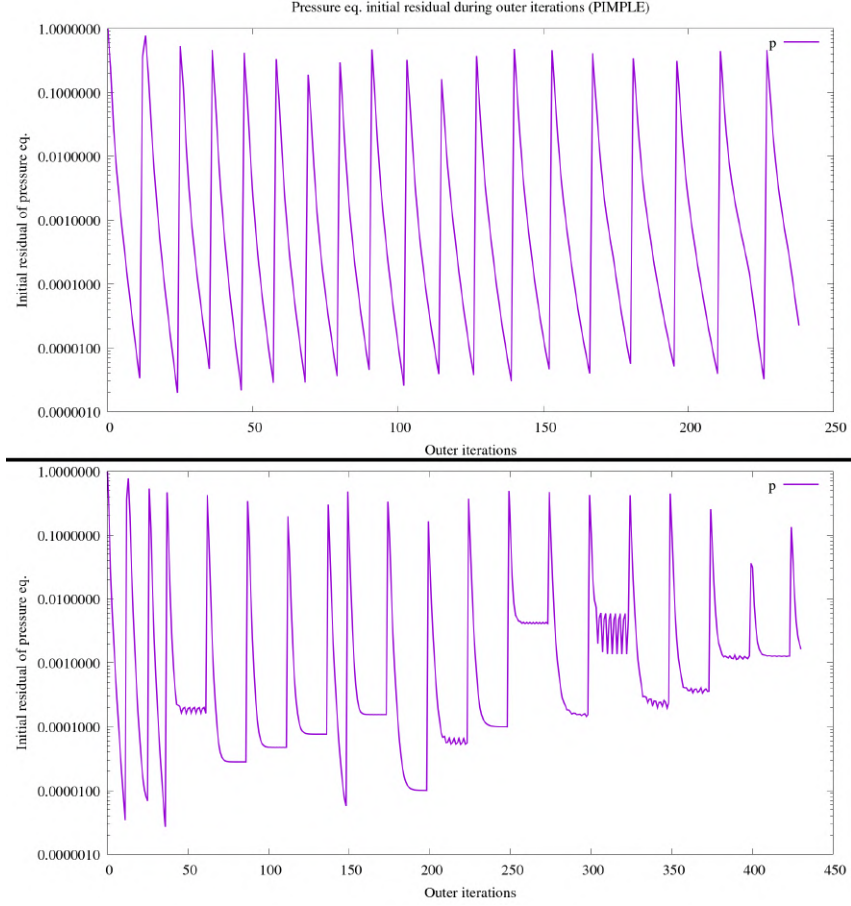


Figure 3.3: initial residuals of the pressure equation for the exact same problem configuration, except for the different divergence scheme for U: linearUpwind (on top) and limitedLinear 1 (on the bottom).

3.2. Characteristic curve

Considering only mesh 1, three simulations were carried out at rotational speeds of 20, 30, and 40 rpm (see cell-data plots of all fields in Appendix A), with the inlet velocity held constant at 1 m/s. The torque coefficient (3.1) curves for these cases are shown in Figure 3.4, considering only the first simulated turbine cycle.

From the data displayed in Figure 3.4 it was possible to compute the average torque coefficient over the turbine cycle, summarized in Table 3.1 and shown in Figure 3.5.

Turbine RPM	20	30	40
Torque coeff. C_T - CFD	0.700	0.7343	0.6521

Table 3.1: numerically obtained characteristic curve of the MRL tidal turbine for a fixed $U_\infty = 1$ m/s.

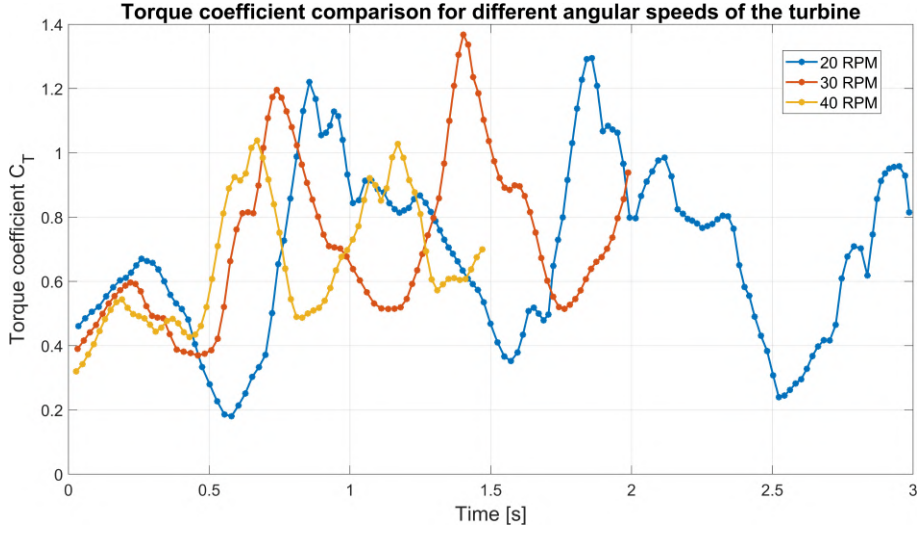


Figure 3.4: trend of the time-averaged torque coefficient for all three turbine rotational speeds considered (20, 30 and 40 rpm).

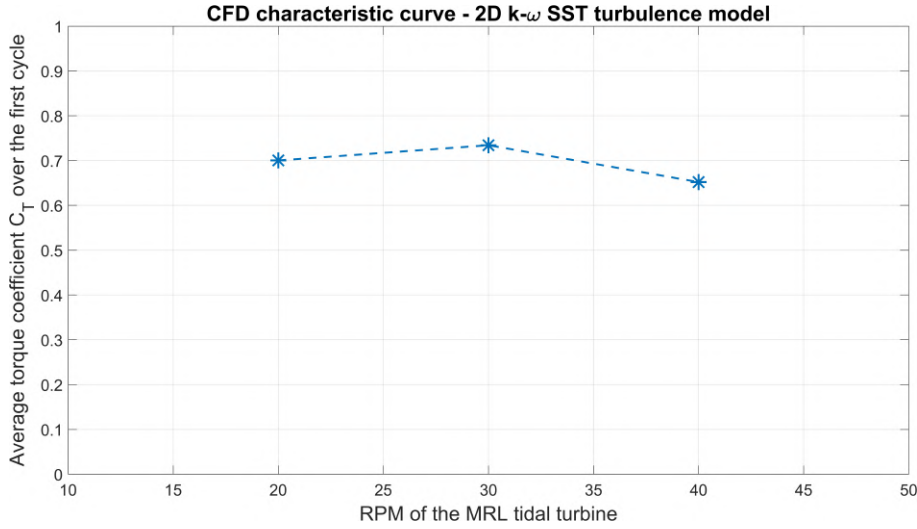


Figure 3.5: numerically obtained characteristic curve of the MRL tidal turbine for a fixed $U_\infty = 1 \text{ m/s}$.

3.3. Grid independency study

Before the grid independence study, it was necessary to check how much the size of the fluid domain influenced the results. Therefore, the results obtained with the first mesh at 40 rpm were compared with the ones obtained with an equivalent mesh and turbine angular speed, but with double the size of the domain. As it's displayed in Figure 3.6, the trends of the time-averaged total torque are compatible, with a relative difference between their average value of 3.6% over the first cycle.

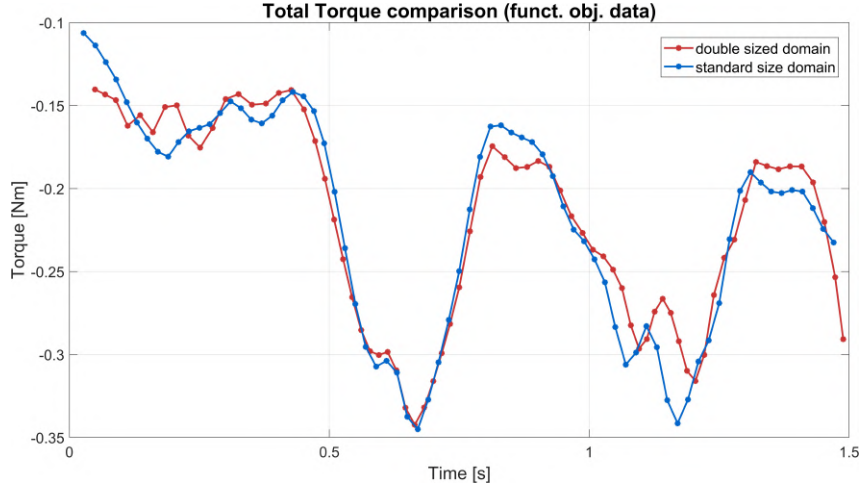


Figure 3.6: comparison between the total torque obtained with the original size domain and the double sized one.

For the grid independency analysis, only the 40 rpm case was considered, the simulation was run with all three meshes described in subsection 2.2.3. The difference in the torque coefficient (3.1) obtained from the different meshes can be observed in Figure 3.7.

$$C_T = \frac{T}{\frac{1}{2}\rho A R U_\infty^2} \quad (3.1)$$

In (3.1) the term A is the frontal area of the turbine, so in this case was the diameter D of the turbine multiplied by the depth of the fluid domain. R is instead the distance between center of the machine and the center of the blades.

Moreover, in Figure 3.8 are displayed the average torque coefficients obtained with the three different mesh refinements.

The relative change in torque coefficient from the coarsest mesh (mesh 3) to the intermediate one (mesh 1) was **-2.81%**, while the relative difference between mesh 1 and the most refined one (mesh 2) was **-0.50%**, meaning that mesh 1 can be practically defined as mesh independent.

3.4. Results comparison

3.4.1. Comparison between numerical schemes

The results obtained for the 40 rpm case, with the first mesh and a second order accurate (in space) discretization of the divergence of velocity field $\vec{U}$, were compared with the ones

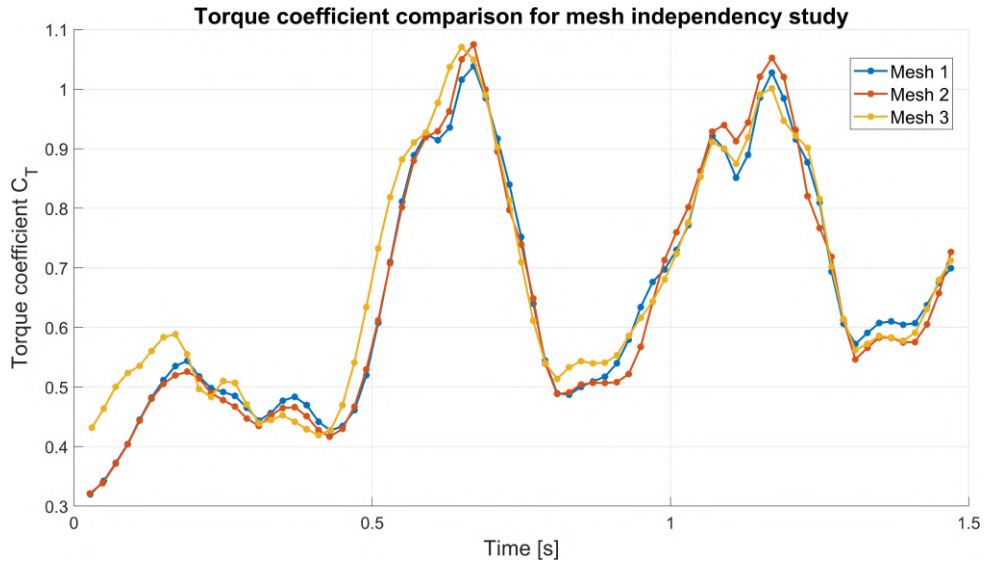


Figure 3.7: comparison between trends of torque coefficient evaluated with the three different meshes.

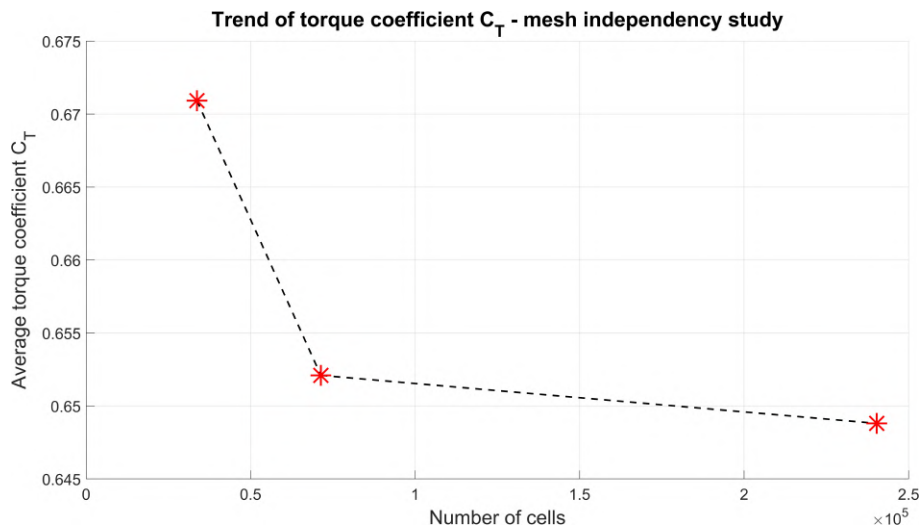


Figure 3.8: average torque coefficient over the first turbine cycle obtained with the three different meshes.

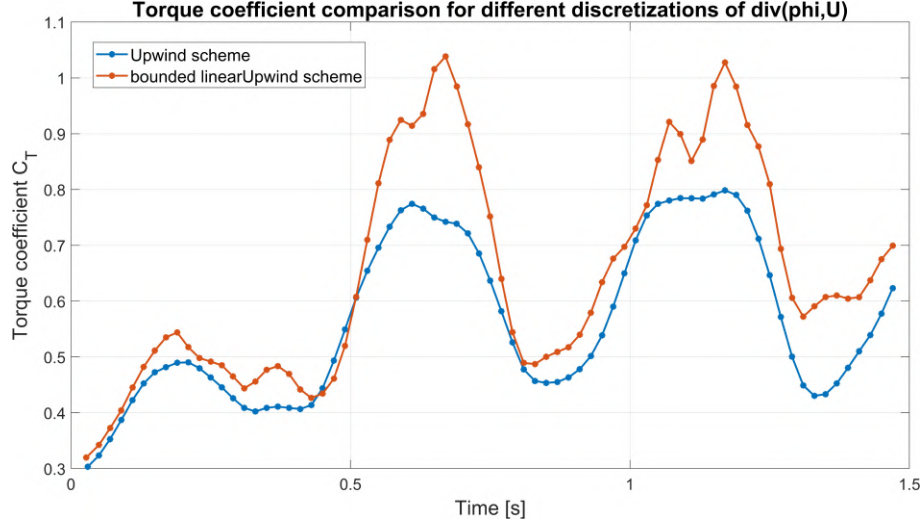


Figure 3.9: comparison between the torque coefficients obtained with different discretizations of the divergence of $\mathbf{U}$.

obtained with the upwind method for the discretization of the same term (Figure 3.9).

The upwind 1st order accurate method is typically not used for the discretization of the divergence of $\bar{\mathbf{U}}$ due to its low accuracy, in fact, in this case the averaged torque coefficient over the first cycle was underestimated by **13.4%** with respect to the higher order numerical scheme, a non-negligible variation between the two.

3.4.2. Comparison between turbulence models

As stated in section 2.1, the $k - \omega SST$ turbulence model was the first choice for this project, but Spalart-Allmaras was not completely discarded. The quality of these two models is compared in Figure 3.10, where the same 40 rpm case is taken into consideration.

These two turbulence models share similar trend in their results but with some non-negligible discrepancies, obtaining a relative difference in the averaged torque coefficient of **5.0%** for mesh 1 and **6.7%** when considering mesh 2. The results obtained with Spalart-Allmaras model were always closer to the experimental results compared to $k - \omega SST$.

3.4.3. Comparison between numerical and experimental results

To evaluate the accuracy and predictive capability of the CFD model, the characteristic curve obtained from the numerical simulation was compared with the experimentally measured data [1]. Figure 3.11 presents a direct comparison between these characteristic

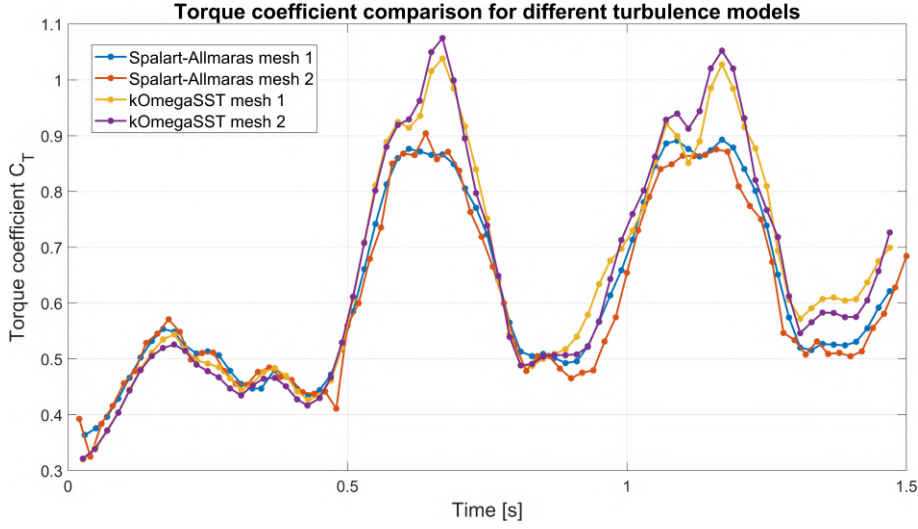


Figure 3.10: comparison between the torque coefficients obtained with $k - \omega SST$ and S-A turbulence models for both mesh 1 and mesh 2.

curves. As it's noticeable, the CFD torque curve presents an earlier peak compared to the one ($\geq 45 \text{ rpm}$) found experimentally and from the manufacturer. The discrepancy goes from around **+24%** to as high as **87%** in the analyzed rotational speed range (20-40 rpm). Finally, the smoothed velocity upstream and downstream from the turbine, along the flow direction, is displayed in Figure 3.12 where it's compared to the experimental one [1].

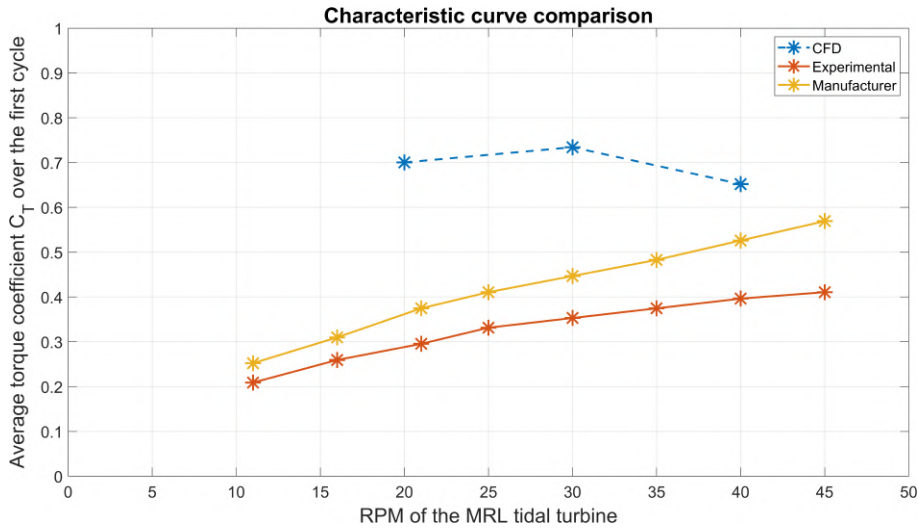


Figure 3.11: comparison between numerical and experimental characteristic curves.

Several factors may explain the discrepancy observed between the characteristic curves. First, the 2D numerical approach does not capture the 3D nature of the turbulent flow around the turbine, such as blade tip vortices. Additionally, the experimental boundary

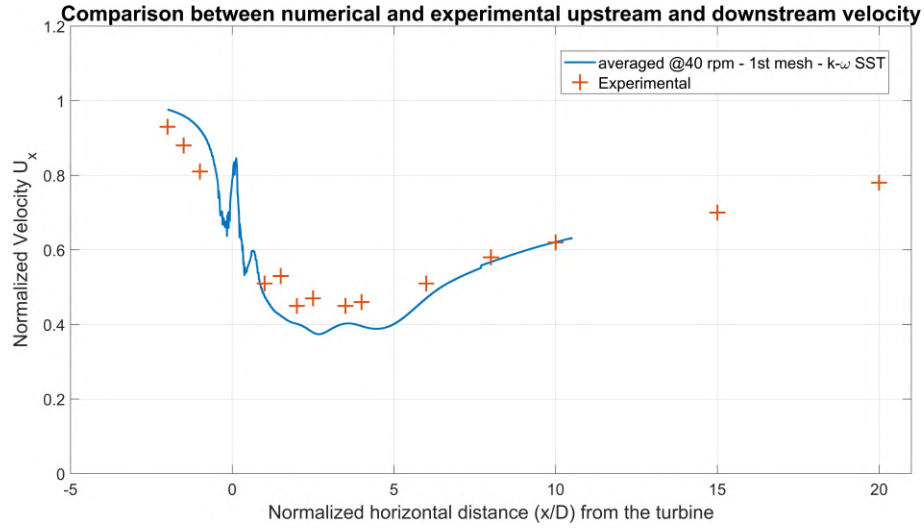


Figure 3.12: comparison between numerical and experimental velocity upstream and downstream from the turbine.

conditions (fluid machine close to the free surface and relatively close to the bottom of the pool) were simplified. Another possible source of error could have been the maximum aspect ratio of the mesh achieved (30-50), which at the blades leading and trailing edges increased the non-orthogonality of the layered mesh, in a fundamental region for the description of flow separation. The pressure field close to the NCC interfaces was, for some time-steps, unphysical, as shown in Figure 3.13. This kind of instabilities may have been generated by the interpolation error across the NCC.

The following solutions were attempted to investigate some of these issues, but with no successful outcome:

- let run the simulations for more than one revolution of the turbine (Figure 2.10);
- start the simulation without the fields obtained from the steady-state solution and let it run for more than one turbine cycle;
- use of Spalart-Allmaras turbulence model for different rotational speed of the machine.
- refine the NCC interface (already a layered mesh) to minimize the spurious values of pressure (Figure 3.13) or just increase the diameter of the region containing each blade to not affect the pressure distribution on the blade leading and trailing edge.

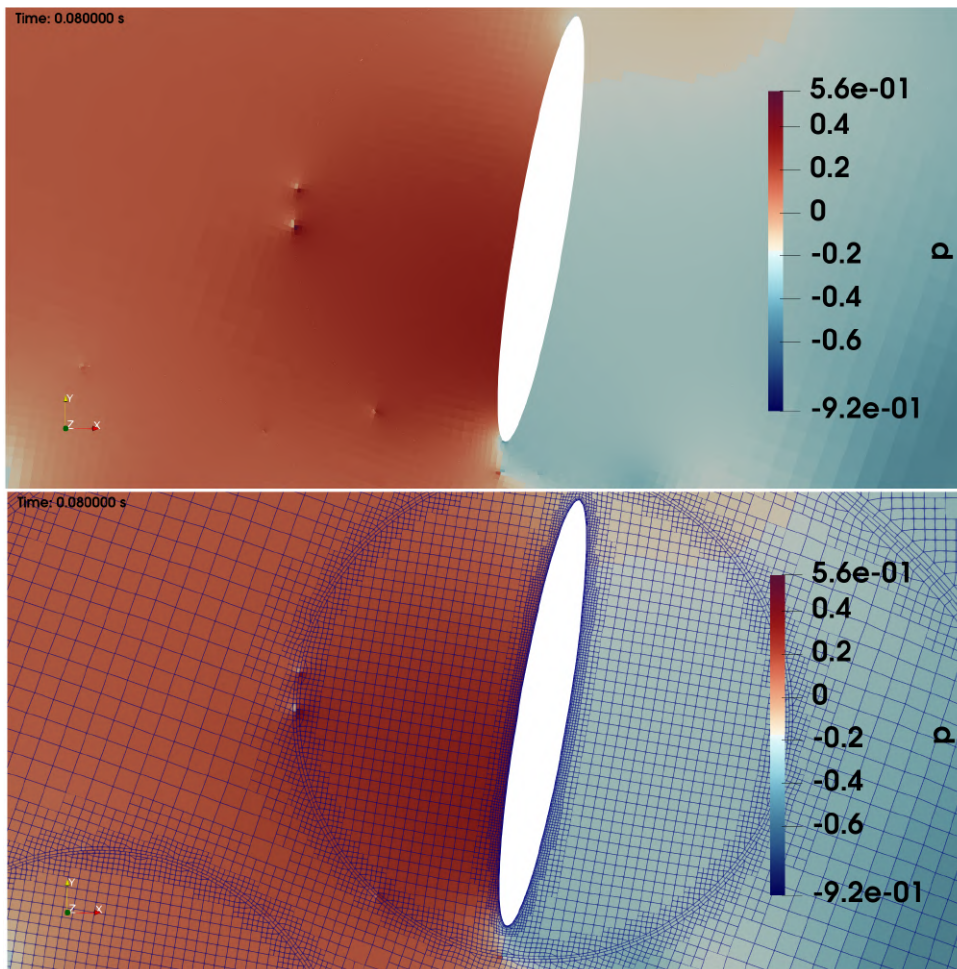


Figure 3.13: nonphysical values of the pressure field localized on cell couples close to the NCC interface.

4 | Conclusions

This study presented a comprehensive CFD investigation of a Momentum Reversal and Lift (MRL) tidal turbine using a 2D unsteady RANS approach. The simulations employed the $k-\omega SST$ and Spalart-Allmaras turbulence models, with careful attention to mesh quality, numerical scheme accuracy, and physical modeling choices. A grid independence study and sensitivity analyses on turbulence models and discretization schemes were performed to validate the robustness of the numerical results.

The key findings include:

- Mesh 1 was demonstrated to be practically grid-independent.
- Among the turbulence models, Spalart-Allmaras gave results closer to experimental data than $k-\omega SST$, though both failed to capture the experimental trend in torque coefficient.
- A discrepancy going from +24% up to 87% was observed between numerical and experimental torque coefficients, likely due to a combination of 2D modeling simplification, turbulence model limitations, simplified boundary conditions and relatively large aspect ratio at the leading and trailing edges.

Despite efforts to mitigate these issues, such as altering initialization methods, using different turbulence models, letting the simulation reach a steady periodic trend and refining the NCC interface, the numerical results remained inconsistent with experimental trends.

To address these discrepancies, future work could involve 3D CFD simulations across varying RPMs to better capture C_T -RPM trends, followed by Volume of Fluid (VoF) CFD modeling to account for free-surface effects if needed.

Bibliography

- [1] M. Gebreslassie, S. Sanchez, G. Tabor, M. Belmont, T. Bruce, G. Payne, and I. Moon. "experimental and cfd analysis of the wake characteristics of tidal turbines.". *International Journal of Marine Energy* 16: 209-219, 2016.
- [2] NASA. Spalart–allmaras model. URL <https://turbmodels.larc.nasa.gov/spalart.html>.
- [3] M. Shur, M. Strelets, L. Zajkov, G. AN, K. VE, and A. Secundov. Comparative numerical testing of one- and two-equation turbulence models for flows with separation and reattachment. 01 1995. doi: 10.2514/6.1995-863.
- [4] P. Spalart and S. Allmaras. A one-equation turbulence model for aerodynamic flows. *AIAA*, 439, 01 1992. doi: 10.2514/6.1992-439.
- [5] D. Sutherland, S. Ordonez, M. Belmont, I. Moon, J. Steynor, T. Davey, and T. Bruce. Experimental optimisation of power for large arrays of cross-flow tidal turbines. *Renewable Energy*, 116, 2017.

A | Appendix A

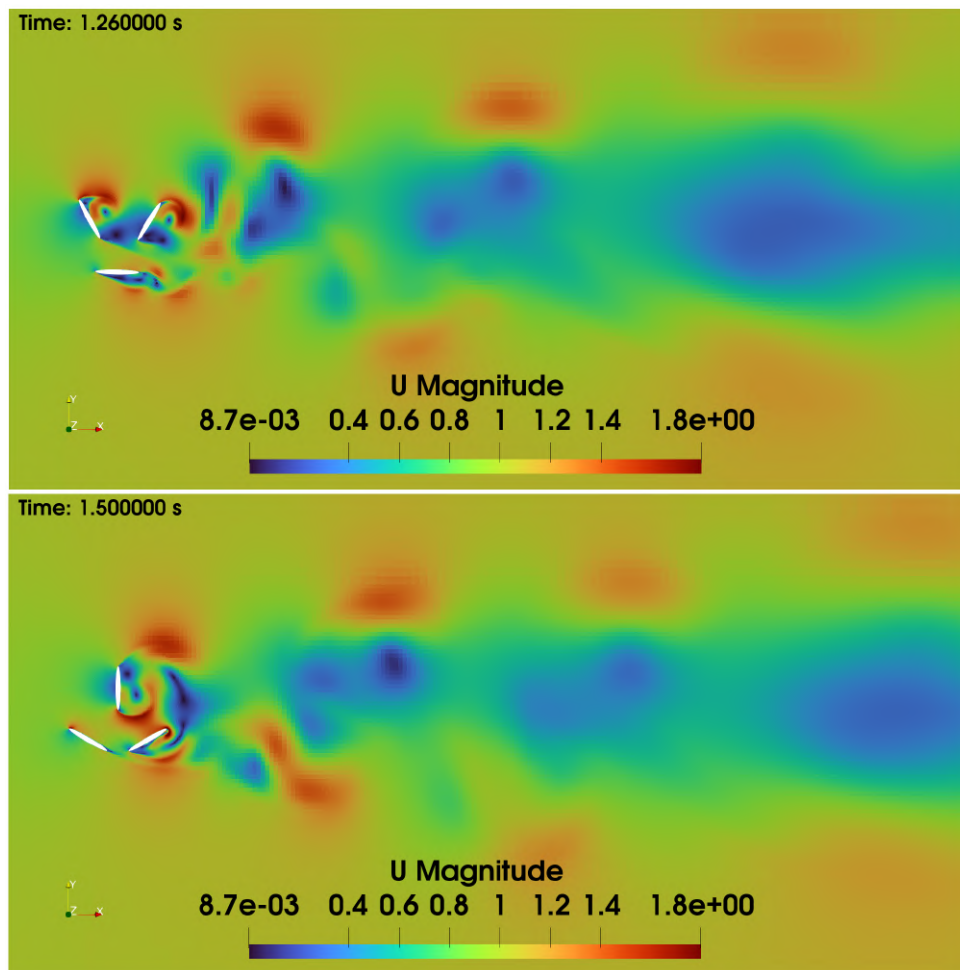


Figure A.1: velocity field obtained with mesh 1 (intermediate refinement), 40 rpm, $k - \omega SST$ turbulence model.

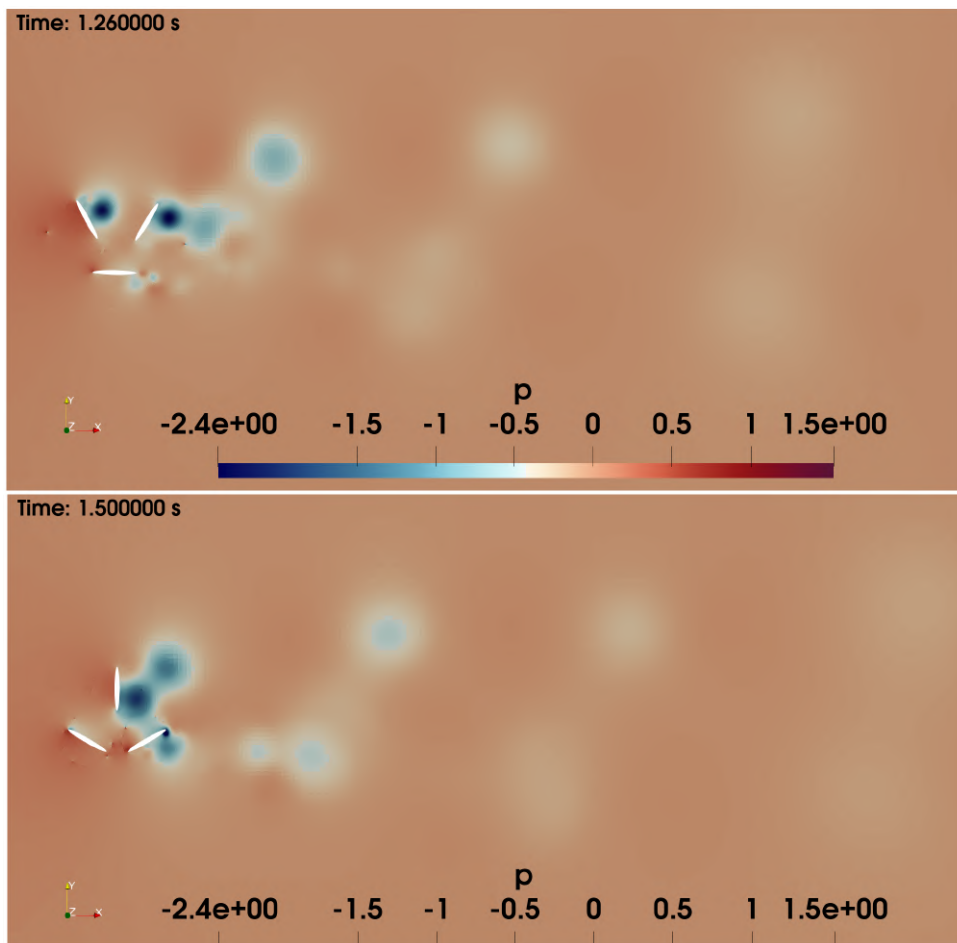


Figure A.2: pressure field obtained with mesh 1 (intermediate refinement), 40 rpm, $k - \omega SST$ turbulence model.

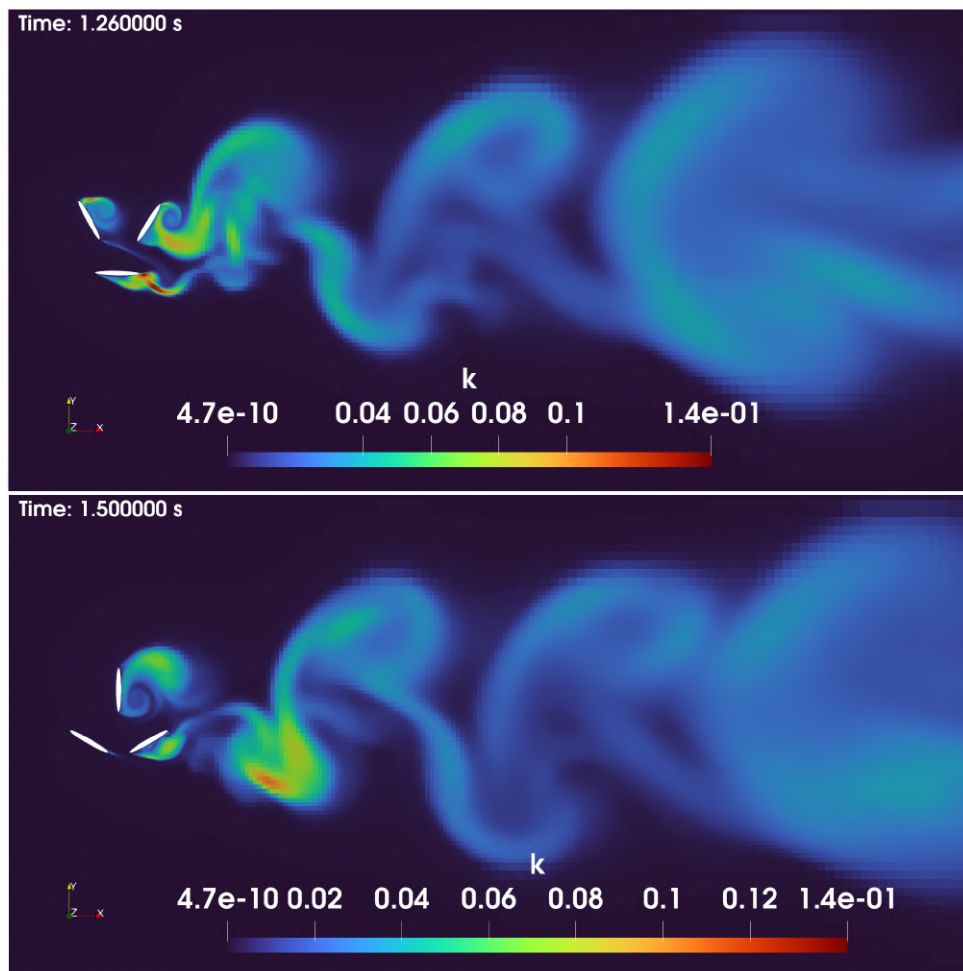


Figure A.3: turbulent kinetic energy field obtained with mesh 1 (intermediate refinement), 40 rpm, $k - \omega SST$ turbulence model.

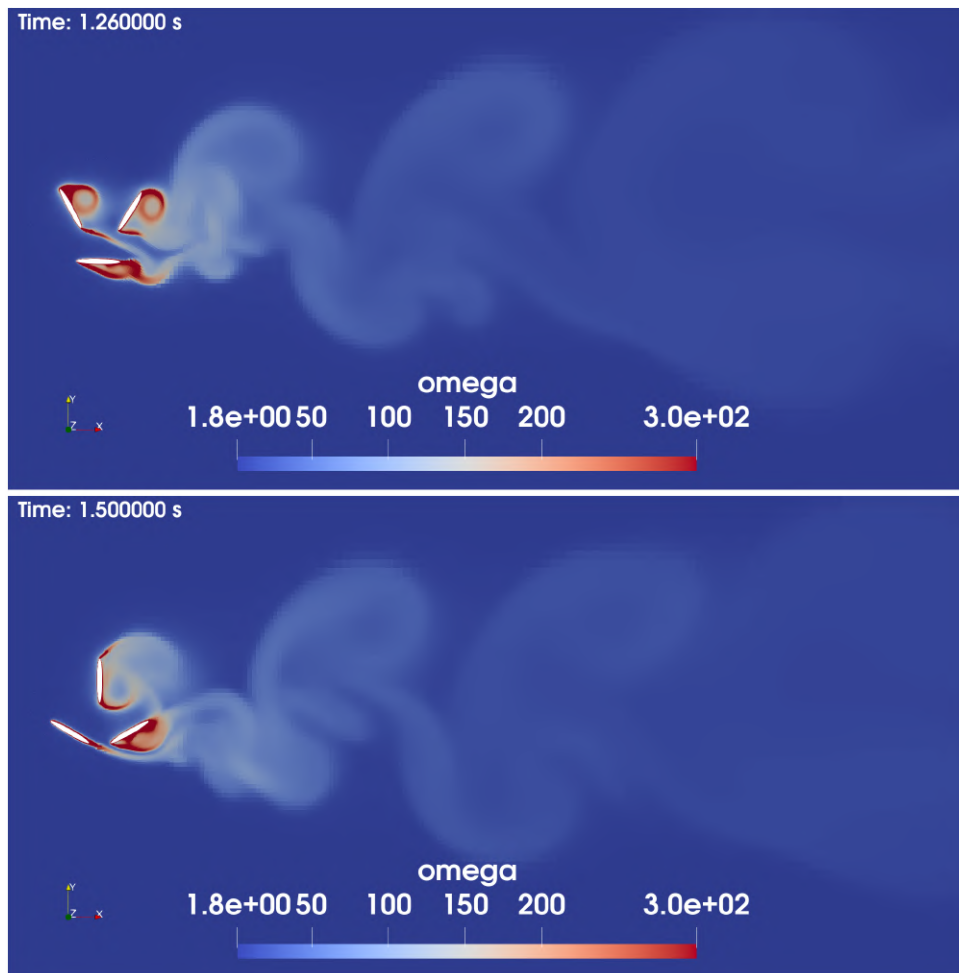


Figure A.4: turbulence dissipation rate field obtained with mesh 1 (intermediate refinement), 40 rpm, $k - \omega SST$ turbulence model.

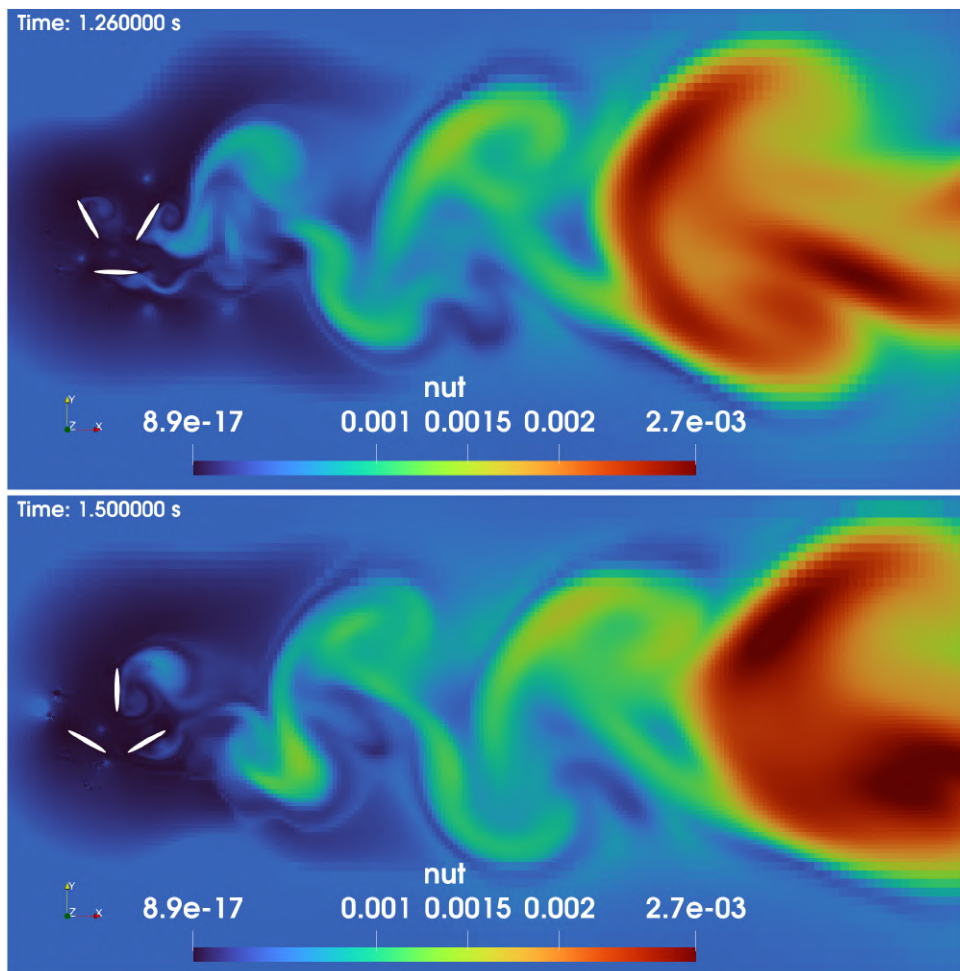


Figure A.5: turbulent viscosity field obtained with mesh 1 (intermediate refinement), 40 rpm, $k - \omega SST$ turbulence model.

